

ОТЧЁТ ПО HW5 “Клиент-серверное приложение на WebSocket и Protocol Buffers”

ФИО: Александр Васюков

Группа: БПИ-235

Email: avvasiukov@edu.hse.ru

Цель работы

Разработать клиент-серверное приложение чата, в котором серверная часть написана на Go с использованием `net.Conn` и `bufio`, клиентская часть написана на чистом JavaScript, транспортом является WebSocket в `binary mode`, а бизнес-протокол описан через Protocol Buffers.

Декларация об использовании ИИ

Я претендую на оценку до 10 баллов, генеративный ИИ не был использован для работы над этим заданием.

Использованные средства

- Язык серверной части: Go 1.26.1
- Сетевые средства сервера: `net.Listener`, `net.Conn`, `bufio`
- Клиент: одна HTML-страница с чистым JavaScript
- Транспорт: WebSocket `binary mode`
- Бизнес-протокол: Protocol Buffers, синтаксис `proto3`
- Форматы изображений: JPEG и PNG до 1 МБ

Архитектура решения

Решение разделено на несколько слоев:

- `domain` содержит правила предметной области: ограничения имени, длины текста, размера изображения, формата изображения и лимитов истории;
- `app` содержит `application service Hub`, который управляет пользователями, историей и маршрутизацией сообщений;
- `websocket` реализует WebSocket handshake и чтение/запись `binary frames` поверх `net.Conn`;
- `server` связывает TCP/HTTP/WebSocket-слой с `application service` и `protobuf`-сообщениями;
- `web/index.html` содержит клиентский интерфейс и кодирование/декодирование `protobuf`-сообщений на чистом JavaScript.

```
HW-05/
├── cmd/server/main.go
├── internal/
│   ├── app/
│   ├── config/
│   ├── domain/
│   ├── protocol/chatpb/
│   ├── server/
│   └── websocket/
├── proto/chat.proto
├── web/index.html
├── README.md
└── report.typ
```

Бизнес-протокол на Protocol Buffers

Схема находится в `proto/chat.proto`. Используется `proto3`.

Основные сообщения:

- `ClientRequest` — общий запрос клиента;
- `ServerResponse` — общий ответ сервера;
- `JoinRequest` — запрос на подключение к чату;
- `SendMessageRequest` — запрос отправки сообщения;
- `ChatMessage` — сообщение чата;
- `History` — история сообщений, изображений и пользователей;
- `Image` — бинарное изображение;
- `User` и вложенное сообщение `User.Profile`.

В протоколе используются:

- несколько типов полей: `string`, `bytes`, `int32`, `int64`, `bool`;
- `optional` поля: `icon`, `to`, `image`, `last_image`, `error`, `history`, `message`;
- `repeated` поля: `History.messages`, `History.images`, `History.users`;
- вложенное сообщение: `User.Profile`;
- `default`-значения `proto3`, например `bool private = false`, `bool request_history = false`, `int32 max_history = 0`.

На сервере все `optional`-варианты проверяются явно: `GetJoin()`, `GetSend()`, `GetImage()`, `GetTo()`, `GetIcon()`.

WebSocket binary mode

WebSocket реализован вручную поверх `net.Conn`:

1. Сервер читает HTTP-запрос через `http.ReadRequest`.
2. Для пути `/ws` проверяется `Upgrade: websocket`.
3. Сервер вычисляет `Sec-WebSocket-Accept`.
4. Далее соединение переводится в режим `WebSocket`.
5. Клиентские frames читаются с `mask`, серверные `binary frames` отправляются без `mask`, как требует `WebSocket`.
6. Payload каждого `binary frame` — `protobuf`-сообщение.

7. Для больших сообщений поддерживаются `fragmented frames` и `continuation frames`, поэтому JPEG/PNG передаются как единое `protobuf`-сообщение даже при фрагментации браузером.

Сервер также обрабатывает `ping` и `close frames`.

Сценарий подключения

Пользователь вводит имя и нажимает Присоединиться. После этого клиент открывает WebSocket-соединение и отправляет `JoinRequest`.

Сервер:

- проверяет, что имя непустое;
- проверяет, что имя еще не занято;
- проверяет лимит 100 пользователей;
- сохраняет `icon` пользователя, если он передан;
- возвращает `ServerResponse` с историей.

Если имя занято, возвращается `ServerResponse` с `ok = false` и сообщением об ошибке.

Отправка сообщений

Обычное сообщение отправляется всем активным клиентам, включая отправителя.

Приватное сообщение вводится в формате:

@name текст

В этом случае клиент заполняет поле `to`, а сервер отправляет сообщение только получателю и отправителю. Если получателя нет в чате, сервер возвращает ошибку.

Работа с изображениями

К сообщению можно прикрепить изображение JPEG/PNG до 1 МБ. Изображение передается в `protobuf`-сообщении `Image`, где есть:

- `mime_type`;
- `data`;
- `size_bytes`.

Если изображение прикреплено при подключении, оно используется как `icon` пользователя и отправляется вместе с последующими сообщениями этого пользователя.

История

Сервер хранит:

- до 50 последних сообщений;
- до 50 последних изображений.

При подключении нового пользователя сервер отправляет историю сообщений и последнее изображение при наличии.

Запуск

```
cd HW-05  
go run ./cmd/server
```

Открыть в браузере:

```
http://127.0.0.1:8080
```

Проверка

```
cd HW-05  
go test ./...
```

Проверены:

- запрет одинаковых имен;
- broadcast-сообщения;
- ошибка при приватном сообщении отсутствующему получателю;
- ограничение истории последними 50 сообщениями.

Вывод

В работе реализован многопользовательский чат с WebSocket binary mode и бизнес-протоколом на Protocol Buffers. Сервер работает поверх `net.Conn` и `bufio`, клиент не использует JavaScript-фреймворки. Реализованы подключение по имени, история, изображения, `icon` пользователя, приватные сообщения и обработка ошибок.